VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

CHENNAI

NAME OF THE STUDENT:	Diraq Najir Girach
REGISTER NUMBER:	23BCE1112
COURSE CODE:	BCSE309P
COURSE TITLE:	Cryptography and Network Security Lab
CLASS NUMBER:	CH2025260501999
SLOT:	L15+L16
COURSE HANDLING FACULTY:	Prof. Karthika V

DIGITAL SIGNATURE STANDARD (DSS)

IMPLEMENTATION

SENDER (CLIENT)

```java
import java.io.*;
import java.net.*;
import java.math.BigInteger;
import java.security.MessageDigest;
import java.security.SecureRandom;
import java.util.*;

public class sender {
    public static void main(String[] args) throws Exception {
        SecureRandom random = new SecureRandom();

        // 1. Generate DSS Parameters
        BigInteger q = new BigInteger(160, 50, random); // 160-bit prime

        BigInteger p, kMultiplier;
        do {
            kMultiplier = new BigInteger(512 - 160, random);
            p = kMultiplier.multiply(q).add(BigInteger.ONE);
        } while (!p.isProbablePrime(50)); // 512-bit prime

        BigInteger h, g;
        do {
            h = new BigInteger(511, random);
            g = h.modPow(p.subtract(BigInteger.ONE).divide(q), p);
        } while (g.compareTo(BigInteger.ONE) <= 0); // Generator

        // 2. Generate Keys
        BigInteger x = new BigInteger(159, random); // Private key
        BigInteger y = g.modPow(x, p);              // Public key

        // 3. Define the Message
        System.out.println("enter message:");
        Scanner in = new Scanner(System.in);
        String message = in.nextLine();
        System.out.println("select 1 to simulate authentic message, 0 for otherwise:");
        int ch = in.nextInt();
        if(ch == 0){
            x = x.add(BigInteger.ONE);
        }

        System.out.println("--- Sender (Client) ---");
        System.out.println("Message: " + message);

        // 4. Call the explicit sign function
        // Returns an array where index 0 is 'r' and index 1 is 's'
        BigInteger[] signature = sign(message, p, q, g, x);
        BigInteger r = signature[0];
        BigInteger s = signature[1];

        System.out.println("Generated Signature (r): " + r);
        System.out.println("Generated Signature (s): " + s);
```

```
        // 5. Connect to Receiver and transmit data
        try (Socket socket = new Socket("localhost", 5000);
             ObjectOutputStream out = new ObjectOutputStream(socket.getOutputStream())) {

            out.writeObject(p);
            out.writeObject(q);
            out.writeObject(g);
            out.writeObject(y);
            out.writeObject(message);
            out.writeObject(r);
            out.writeObject(s);

            System.out.println("\nPayload securely sent to the receiver.");
        }
    }

    public static BigInteger[] sign(String message, BigInteger p, BigInteger q, BigInteger g,
BigInteger x) throws Exception {
        SecureRandom random = new SecureRandom();

        // Hashing the message (SHA-256)
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        BigInteger hash = new BigInteger(1, md.digest(message.getBytes()));

        BigInteger k, r, s;
        do {
            // Random per-message key (0 < k < q)
            k = new BigInteger(159, random);

            // r = (g^k mod p) mod q
            r = g.modPow(k, p).mod(q);

            // s = (k^-1 * (H(m) + x*r)) mod q
            BigInteger kInv = k.modInverse(q);
            s = kInv.multiply(hash.add(x.multiply(r))).mod(q);

        // Ensure neither r nor s are zero (extremely rare, but required by DSS standard)
        } while (r.equals(BigInteger.ZERO) || s.equals(BigInteger.ZERO));

        return new BigInteger[]{r, s};
    }
}
```

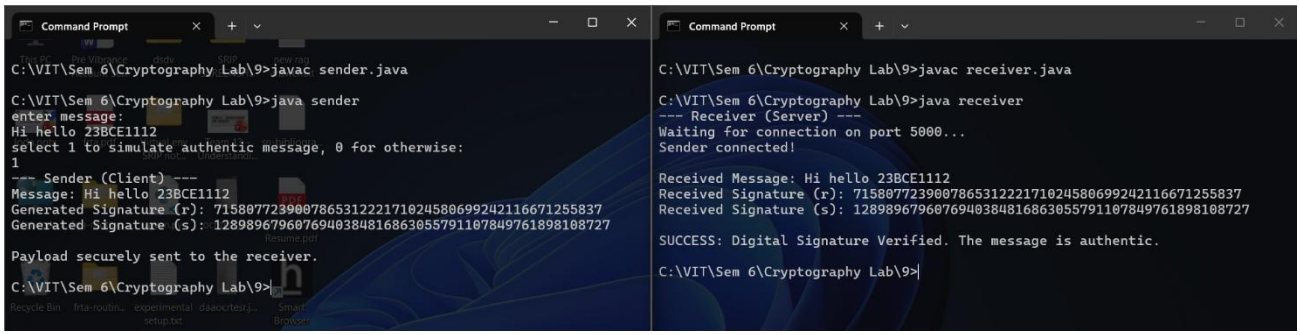
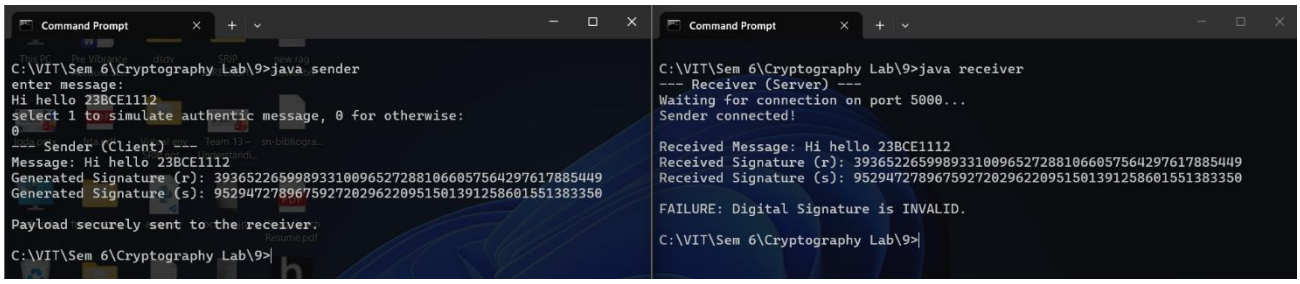
RECEIVER (SERVER)

```
import java.io.*;
import java.net.*;
import java.math.BigInteger;
import java.security.MessageDigest;

public class receiver {
    public static void main(String[] args) throws Exception {
        System.out.println("--- Receiver (Server) ---");
        System.out.println("Waiting for connection on port 5000...");

        // Start server socket
        try (ServerSocket serverSocket = new ServerSocket(5000);
             Socket socket = serverSocket.accept();
             ObjectInputStream in = new ObjectInputStream(socket.getInputStream())) {

            System.out.println("Sender connected!\n");

            // 1. Receive DSS parameters, public key, message, and signature
            BigInteger p = (BigInteger) in.readObject();
            BigInteger q = (BigInteger) in.readObject();
            BigInteger g = (BigInteger) in.readObject();
            BigInteger y = (BigInteger) in.readObject();
            String message = (String) in.readObject();
            BigInteger r = (BigInteger) in.readObject();
            BigInteger s = (BigInteger) in.readObject();

            System.out.println("Received Message: " + message);
            System.out.println("Received Signature (r): " + r);
            System.out.println("Received Signature (s): " + s);

            // 2. Verify Signature
            boolean isValid = verify(p, q, g, y, message, r, s);

            if (isValid) {
                System.out.println("\nSUCCESS: Digital Signature Verified. The message is
authentic.");
            } else {
                System.out.println("\nFAILURE: Digital Signature is INVALID.");
            }
        }
    }

    // Standard DSS Verification Logic
    public static boolean verify(BigInteger p, BigInteger q, BigInteger g, BigInteger y,
                                 String message, BigInteger r, BigInteger s) throws Exception {
        // Step A: Checking bounds 0 < r < q and 0 < s < q
        if (r.compareTo(BigInteger.ZERO) <= 0 || r.compareTo(q) >= 0 ||
            s.compareTo(BigInteger.ZERO) <= 0 || s.compareTo(q) >= 0) {
            return false;
        }

        // Step B: Hashing the message (SHA-256)
        MessageDigest md = MessageDigest.getInstance("SHA-256");
        BigInteger hash = new BigInteger(1, md.digest(message.getBytes()));

        // Step C: w = s^-1 mod q
```

```java
        BigInteger w = s.modInverse(q);

        // Step D: u1 = (H(m) * w) mod q
        BigInteger u1 = hash.multiply(w).mod(q);

        // Step E: u2 = (r * w) mod q
        BigInteger u2 = r.multiply(w).mod(q);

        // Step F: v = ((g^u1 * y^u2) mod p) mod q
        BigInteger v1 = g.modPow(u1, p);
        BigInteger v2 = y.modPow(u2, p);
        BigInteger v = v1.multiply(v2).mod(p).mod(q);

        // Step G: Signature is valid if v == r
        return v.equals(r);
    }
}
```

OUTPUT

VALID CASE



INVALID CASE